Dinesh Arunachalam Robotics Software Engineer

Chennai, Tamil Nadu, India work.dinesh116@gmail.com github.com/dinesh1160
+917305072140 linkedin.com/in/dinesh1160 dinesh1160.github.io/portfolio

Summary

Robotics software engineer (AI & Robotics) building perception, embedded and AI-driven systems for real-world robots. Hands-on across sensing, firmware, ROS 2 software and hardware integration, with a focus on physical AI.

Education

B.Tech Computer Science Engineering (Specialization in AI & Robotics) 07/2023 - Expected 05/2027 | Chennai
Vellore Institute of Technology
CGPA: 8.8/10

Professional Experience

Robotics Software Engineer (Intern), *Sentrix Robotics* 02/2026 - 04/2026 | Chennai
Physical AI startup for robotic dexterity

- Integrated a motion-capture glove with a robotic arm for precise, real-time teleoperation
- Developed C++ firmware on ESP32-C6 and ROS 2 middleware over serial, mapping 5 glove sensor channels to arm joint commands with low latency.
- Integrated a custom PCB with the glove, interfacing sensors over I2C and UART.
- Set up a ROS 2 Humble arm simulation in NVIDIA Isaac Sim (URDF) for validation.
- Extended the glove into a VR input device, using an attached Meta VR controller for spatial tracking and the glove itself for click inputs.

Projects

BEVNav: Camera Only Indoor Navigation,
ROS 2 Jazzy, Gazebo Harmonic, Python, OpenCV, NumPy, PyTorch, robot_localization

- Built a ROS 2 navigation stack for a skid-steer rover with one pan mounted RGB camera: floor masks are projected to a bird's-eye-view grid, six 60° scans are fused into a 360° occupancy grid, and a persistent 50×50 m log-odds map is kept, Masks come from the simulator or a SegFormerB0 model fine-tuned on simulated data.
- Implemented a cost map, greedy planner, 20 Hz pure-pursuit controller and route-based sub-goals so the rover can back out of dead ends, with EKF localization (robot_localization) and REP 105 frames.
- Built an evaluation harness (seeded trials recorded as MCAP bags, automated metrics and figures) over 6 warehouse scenarios, with unit and launch tests. A 9-waypoint tour mapped 92.6% of the reachable floor with no collisions; BEV generation takes 6.1 ms per frame.

Active Perception for a Mobile Robot,
ROS 2 Jazzy, Python, YOLOv8, OpenCV, Gemini VLM, BeetleBot

- Built a ROS 2 pipeline with custom messages and services that scores a target's view using four local metrics (sharpness, exposure, framing, YOLOv8 confidence) and picks a corrective move (retake, orbit, rotate, approach, back off) for the weakest factor.
- Added a closed-loop retry layer that re-scores after each move and retraces to the best view after a fixed number of attempts. A Gemini VLM judges occlusion and framing only after local checks pass, which reduces API calls. Validated end to end in ROS 2 with simulated motion; snapshot pipeline and orbit motion run on the physical BeetleBot.
- Evaluated on 76 controlled degradations: the expected move was chosen in 17 of 22 locally flagged cases (77%), with failure modes documented and unit tests on the metrics and decision logic.

Skills

- **Languages and Tools:** C++, Python, C, SQL, Linux, Git, Docker, CMake, SQLite, C#, Go
- **Robotics and Computer Vision:** ROS 2 (Humble, Jazzy), Gazebo, NVIDIA Isaac Sim, MuJoCo, RViz, URDF/Xacro, tf2, Nav2, robot_localization (EKF), rosbag2/MCAP, SLAM
- **Computer Vision and AI:** OpenCV, YOLOv8, ByteTrack, SegFormer, PyTorch, Vision Language Models, Large Language Models, LangChain, Retrieval-Augmented Generation (RAG)
- **Embedded Systems and Firmware:** ESP32, Arduino, Raseberry Pi, IMU, Flex Sensors, LiDAR
- **Relevant Coursework:** Robot Kinematics and Dynamics, Robotics Perception, Robot Progamming, Artificial Intelligence, Machine Learning, Linear Algebra and Multivariable Calculus